

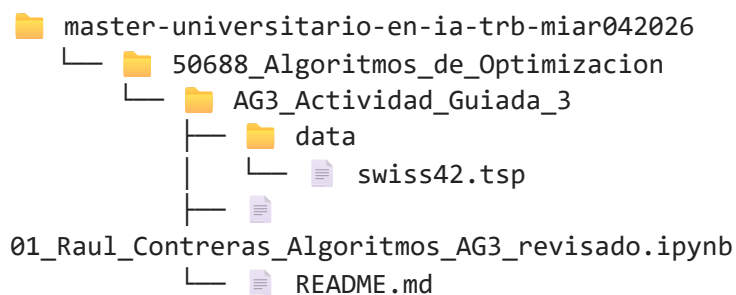
Algoritmos de optimización - Colonia de Hormigas

- **Nombre y Apellidos:** Raúl Contreras O.
- ****Grupo:**

Url:

[Git] https://github.com/raulcontreraso/master-universitario-en-ia-trb-miar042026/tree/main/50688_algoritmos_de_optimizacion/AG3_actividad_Guiada_3

Ruta: master-universitario-en-ia-trb-miar042026/50688_algoritmos_de_optimizacion/AG3_actividad_Guiada_3/



Problema: AG3 - Actividad Guiada 3

1. Desarrollar algoritmos con la técnica de búsqueda aleatoria
2. Desarrollar algoritmos con la técnica de búsqueda local
3. Desarrollar algoritmos con la técnica de recocido simulado(simulated annealing)
4. Desarrollar algoritmos con la técnica de colonia de hormigas(no evaluable)

Carga de librerías

```
In [1]: #Update
        #!python -m pip install --upgrade pip

        #Hacer llamadas http a paginas de la red
        #!pip install requests

        #Modulo para las instancias del problema del TSP
        #!pip install tsplib95

        !python -m pip install requests
        !python -m pip install tsplib95
```


[illegible]

[illegible]

```
----- 5/6 [tsplib95]
----- 5/6 [tsplib95]
----- 6/6 [tsplib95]
```

Successfully installed Click-8.4.2 Deprecated-1.2.18 networkx-2.8.8 tabulate-0.8.10 tsplib95-0.7.1 wrapt-1.17.3

```
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
WARNING: Cache entry deserialization failed, entry ignored
```

Carga de los datos del problema

```
In [2]: # Importar librerías
print("¡Importando Librerías!")
import math
import random
import copy
import numpy
import sys

import urllib.request #Hacer llamadas http a paginas de la red
import tsplib95        #Modulo para las instancias del problema del TSP
import math            #Modulo de funciones matematicas. Se usa para exp
import random          #Para generar valores aleatorios
import requests
import tsplib95
import os

print("¡Librerías importadas correctamente!")

import numpy as np

print(f"Python Executable: {sys.executable}")
print(f"NumPy Version: {np.__version__}")
```

```
¡Importando Librerías!
¡Librerías importadas correctamente!
Python Executable: C:\Users\raulc\AppData\Local\Programs\Python\Python314\python.exe
NumPy Version: 2.4.4
```

```
In [3]: #http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
#Documentacion :
# http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
# https://tsplib95.readthedocs.io/en/stable/pages/usage.html
# https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
# https://pypi.org/project/tsplib95/

#Descargamos el fichero de datos(Matriz de distancias)
```

```

#file = "swiss42.tsp" ; ##### Leer es
# urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/swiss42.tsp.gz", i
# !gzip -d swiss42.tsp.gz #Descomprimir el fichero de datos

#URL alternativa
# file = "swiss42.tsp" ;
# urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/heads/master/swiss4

#Coordendas 51-city problem (Christofides/Eilon)
#file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp

#Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
#file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp

```

```

In [4]: # Se lee el Archivo local
print("¿Existe el archivo?:", os.path.exists("datos/swiss42_.tsp"))
# Ruta exacta donde guardaste el archivo
file = "datos/swiss42_.tsp"

```

¿Existe el archivo?: True

```

In [5]: #Carga de datos y generación de objeto problem
#####
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Aristas
Aristas = list(problem.get_edges())

```

```

In [6]: # Comprobación de que funcionó
print("¡Archivo leído con éxito desde la carpeta data!")
print("Nombre:", problem.name)
print("Dimensión:", problem.dimension)
print("Tipo de pesos:", problem.edge_weight_type)
print(vars(problem))

```

¡Archivo leído con éxito desde la carpeta data!

Nombre: swiss42

Dimensión: 42

Tipo de pesos: EXPLICIT

```
{'name': 'swiss42', 'type': 'TSP', 'comment': '42 Staedte Schweiz (Fricker)', 'dimension': 42, 'edge_weight_type': 'EXPLICIT', 'edge_weight_format': 'FULL_MATRIX', 'edge_weights': [[0, 15, 30, 23, 32, 55, 33, 37, 92, 114, 92, 110, 96, 90, 74, 76, 82, 67, 72, 78, 82, 159, 122, 131, 206, 112, 57, 28, 43, 70, 65, 66, 37, 103, 84, 125, 129, 72, 126, 141, 183, 124], [15, 0, 34, 23, 27, 40, 19, 32, 93, 117, 88, 100, 87, 75, 63, 67, 71, 69, 62, 63, 96, 164, 132, 131, 212, 106, 44, 33, 51, 77, 75, 72, 52, 118, 99, 132, 132, 67, 139, 148, 186, 122], [30, 34, 0, 11, 18, 57, 36, 65, 62, 84, 64, 89, 76, 93, 95, 100, 104, 98, 57, 88, 99, 130, 100, 101, 179, 86, 51, 4, 18, 43, 45, 95, 45, 115, 93, 152, 159, 100, 112, 114, 153, 94], [23, 23, 11, 0, 11, 48, 26, 54, 70, 94, 69, 89, 75, 84, 84, 89, 92, 89, 54, 78, 99, 141, 111, 109, 190, 89, 44, 11, 29, 54, 56, 89, 47, 118, 96, 147, 151, 90, 122, 126, 163, 101], [32, 27, 18, 11, 0, 40, 20, 58, 67, 92, 61, 78, 65, 76, 83, 89, 91, 95, 43, 72, 110, 141, 116, 105, 190, 81, 34, 19, 35, 57, 63, 97, 58, 129, 107, 156, 158, 92, 129, 127, 161, 95], [55, 40, 57, 48, 40, 0, 23, 55, 96, 123, 78, 75, 62, 36, 56, 66, 63, 95, 37, 34, 137, 174, 156, 129, 224, 90, 15, 59, 75, 96, 103, 105, 91, 158, 139, 164, 156, 78, 169, 163, 191, 115], [33, 19, 36, 26, 20, 23, 0, 45, 85, 111, 75, 82, 69, 60, 63, 70, 71, 85, 44, 52, 115, 161, 136, 122, 210, 91, 25, 37, 54, 78, 81, 90, 68, 136, 116, 150, 147, 76, 148, 147, 180, 111], [37, 32, 65, 54, 58, 55, 45, 0, 124, 149, 118, 126, 113, 80, 42, 42, 49, 40, 87, 60, 94, 195, 158, 163, 242, 135, 65, 63, 79, 106, 101, 50, 66, 118, 104, 109, 103, 36, 160, 178, 218, 153], [92, 93, 62, 70, 67, 96, 85, 124, 0, 28, 29, 68, 63, 122, 148, 155, 156, 159, 67, 129, 148, 78, 80, 39, 129, 46, 82, 65, 55, 40, 61, 157, 97, 159, 135, 212, 221, 159, 110, 72, 95, 35], [114, 117, 84, 94, 92, 123, 111, 149, 28, 0, 54, 91, 88, 150, 174, 181, 182, 181, 95, 157, 159, 50, 65, 27, 102, 65, 110, 87, 73, 50, 68, 176, 112, 166, 142, 229, 241, 184, 99, 46, 69, 38], [92, 88, 64, 69, 61, 78, 75, 118, 29, 54, 0, 39, 34, 99, 134, 142, 141, 157, 44, 110, 161, 103, 109, 52, 154, 22, 63, 68, 66, 61, 81, 158, 107, 175, 151, 216, 219, 150, 137, 100, 115, 37], [110, 100, 89, 89, 78, 75, 82, 126, 68, 91, 39, 0, 14, 80, 129, 139, 135, 167, 39, 98, 187, 136, 148, 81, 186, 28, 61, 92, 97, 98, 117, 173, 134, 204, 181, 232, 229, 153, 176, 137, 143, 62], [96, 87, 76, 75, 65, 62, 69, 113, 63, 88, 34, 14, 0, 72, 117, 128, 124, 153, 26, 88, 174, 136, 142, 82, 187, 32, 48, 79, 85, 89, 106, 159, 121, 191, 168, 219, 216, 140, 168, 134, 145, 64], [90, 75, 93, 84, 76, 36, 60, 80, 122, 150, 99, 80, 72, 0, 59, 71, 63, 116, 56, 25, 170, 201, 189, 151, 252, 104, 44, 95, 111, 130, 138, 130, 127, 192, 174, 186, 172, 90, 205, 193, 214, 135], [74, 63, 95, 84, 83, 56, 63, 42, 148, 174, 134, 129, 117, 59, 0, 11, 8, 63, 93, 35, 135, 223, 195, 184, 273, 146, 71, 95, 113, 138, 138, 81, 107, 159, 146, 132, 113, 32, 200, 209, 243, 171], [76, 67, 100, 89, 89, 66, 70, 42, 155, 181, 142, 139, 128, 71, 11, 0, 11, 54, 103, 46, 130, 230, 198, 192, 279, 155, 80, 99, 117, 143, 141, 74, 107, 155, 143, 122, 102, 22, 202, 215, 250, 179], [82, 71, 104, 92, 91, 63, 71, 49, 156, 182, 141, 135, 124, 63, 8, 11, 0, 65, 100, 39, 140, 232, 203, 192, 281, 153, 78, 103, 121, 147, 146, 85, 115, 164, 152, 133, 112, 33, 208, 218, 251, 178], [67, 69, 98, 89, 95, 95, 85, 40, 159, 181, 157, 167, 153, 116, 63, 54, 65, 0, 127, 92, 83, 224, 180, 199, 269, 175, 106, 95, 109, 135, 125, 21, 80, 107, 100, 71, 63, 33, 173, 205, 249, 191], [72, 62, 57, 54, 43, 37, 44, 87, 67, 95, 44, 39, 26, 56, 93, 103, 100, 127, 0, 67, 153, 145, 139, 96, 196, 53, 23, 60, 70, 81, 95, 134, 101, 172, 149, 194, 190, 115, 160, 0, 138, 159, 80], [78, 63, 88, 78, 72, 34, 52, 60, 129, 157, 110, 98, 88, 25, 35, 46, 39, 92, 67, 0, 152, 207, 188, 162, 258, 119, 48, 89, 107, 129, 134, 108, 114, 176, 159, 163, 147, 66, 200, 197, 224, 147], [82, 96, 99, 99, 110, 137, 115, 94, 148, 159, 161, 187, 174, 170, 135, 130, 140, 83, 153, 152, 0, 188, 128, 184, 222, 183, 139, 95, 95, 110, 91, 62, 54, 24, 23, 81, 110, 113, 108, 164, 217, 184], [159, 164, 130, 141, 141, 174, 161, 195, 78, 50, 103, 136, 136, 201, 223, 230, 232, 224, 145, 207, 188, 0, 65, 57, 51, 109, 160, 132, 116, 90, 102, 217, 148, 188, 168, 264, 281, 231, 100, 26, 30, 75], [122, 132, 100, 111, 116, 156, 136, 158, 80, 65, 109, 148, 142, 189, 195, 198, 203, 180, 139, 188, 128, 65, 0, 91, 94, 126, 145, 100, 82, 60, 57, 167, 99, 126, 106, 208, 230, 194, 36, 39, 94, 103], [131, 13
```

```

1, 101, 109, 105, 129, 122, 163, 39, 27, 52, 81, 82, 151, 184, 192, 192, 199, 96,
162, 184, 57, 91, 0, 106, 53, 115, 104, 94, 74, 94, 196, 134, 192, 168, 251, 260,
197, 126, 64, 64, 19], [206, 212, 179, 190, 190, 224, 210, 242, 129, 102, 154, 18
6, 187, 252, 273, 279, 281, 269, 196, 258, 222, 51, 94, 106, 0, 158, 211, 180, 16
3, 136, 145, 259, 190, 218, 200, 302, 323, 278, 120, 65, 49, 124], [112, 106, 86,
89, 81, 90, 91, 135, 46, 65, 22, 28, 32, 104, 146, 155, 153, 175, 53, 119, 183, 1
09, 126, 53, 158, 0, 75, 89, 88, 83, 103, 178, 129, 197, 173, 236, 238, 166, 156,
111, 115, 34], [57, 44, 51, 44, 34, 15, 25, 65, 82, 110, 63, 61, 48, 44, 71, 80,
78, 106, 23, 48, 139, 160, 145, 115, 211, 75, 0, 53, 68, 86, 95, 114, 90, 160, 13
9, 173, 168, 92, 162, 150, 176, 101], [28, 33, 4, 11, 19, 59, 37, 63, 65, 87, 68,
92, 79, 95, 95, 99, 103, 95, 60, 89, 95, 132, 100, 104, 180, 89, 53, 0, 18, 44, 4
5, 92, 42, 112, 89, 149, 156, 99, 111, 116, 155, 97], [43, 51, 18, 29, 35, 75, 5
4, 79, 55, 73, 66, 97, 85, 111, 113, 117, 121, 109, 70, 107, 95, 116, 82, 94, 16
3, 88, 68, 18, 0, 27, 27, 103, 42, 109, 85, 157, 168, 115, 94, 98, 140, 90], [70,
77, 43, 54, 57, 96, 78, 106, 40, 50, 61, 98, 89, 130, 138, 143, 147, 135, 81, 12
9, 110, 90, 60, 74, 136, 83, 86, 44, 27, 0, 21, 128, 62, 119, 96, 179, 192, 142,
79, 72, 115, 74], [65, 75, 45, 56, 63, 103, 81, 101, 61, 68, 81, 117, 106, 138, 1
38, 141, 146, 125, 95, 134, 91, 102, 57, 94, 145, 103, 95, 45, 27, 21, 0, 115, 4
6, 98, 75, 163, 179, 136, 67, 81, 129, 95], [66, 72, 95, 89, 97, 105, 90, 50, 15
7, 176, 158, 173, 159, 130, 81, 74, 85, 21, 134, 108, 62, 217, 167, 196, 259, 17
8, 114, 92, 103, 128, 115, 0, 69, 86, 81, 60, 65, 54, 158, 195, 243, 190], [37, 5
2, 45, 47, 58, 91, 68, 66, 97, 112, 107, 134, 121, 127, 107, 107, 115, 80, 101, 1
14, 54, 148, 99, 134, 190, 129, 90, 42, 42, 62, 46, 69, 0, 71, 49, 117, 133, 98,
95, 127, 175, 132], [103, 118, 115, 118, 129, 158, 136, 118, 159, 166, 175, 204,
191, 192, 159, 155, 164, 107, 172, 176, 24, 188, 126, 192, 218, 197, 160, 112, 10
9, 119, 98, 86, 71, 0, 24, 94, 127, 137, 100, 163, 218, 194], [84, 99, 93, 96, 10
7, 139, 116, 104, 135, 142, 151, 181, 168, 174, 146, 143, 152, 100, 149, 159, 23,
168, 106, 168, 200, 173, 139, 89, 85, 96, 75, 81, 49, 24, 0, 104, 133, 127, 85, 1
43, 197, 170], [125, 132, 152, 147, 156, 164, 150, 109, 212, 229, 216, 232, 219,
186, 132, 122, 133, 71, 194, 163, 81, 264, 208, 251, 302, 236, 173, 149, 157, 17
9, 163, 60, 117, 94, 104, 0, 39, 100, 190, 241, 292, 246], [129, 132, 159, 151, 1
58, 156, 147, 103, 221, 241, 219, 229, 216, 172, 113, 102, 112, 63, 190, 147, 11
0, 281, 230, 260, 323, 238, 168, 156, 168, 192, 179, 65, 133, 127, 133, 39, 0, 8
1, 216, 259, 307, 253], [72, 67, 100, 90, 92, 78, 76, 36, 159, 184, 150, 153, 14
0, 90, 32, 22, 33, 33, 115, 66, 113, 231, 194, 197, 278, 166, 92, 99, 115, 142, 1
36, 54, 98, 137, 127, 100, 81, 0, 193, 214, 253, 187], [126, 139, 112, 122, 129,
169, 148, 160, 110, 99, 137, 176, 168, 205, 200, 202, 208, 173, 160, 200, 108, 10
0, 36, 126, 120, 156, 162, 111, 94, 79, 67, 158, 95, 100, 85, 190, 216, 193, 0, 7
4, 129, 137], [141, 148, 114, 126, 127, 163, 147, 178, 72, 46, 100, 137, 134, 19
3, 209, 215, 218, 205, 138, 197, 164, 26, 39, 64, 65, 111, 150, 116, 98, 72, 81,
195, 127, 163, 143, 241, 259, 214, 74, 0, 55, 80], [183, 186, 153, 163, 161, 191,
180, 218, 95, 69, 115, 143, 145, 214, 243, 250, 251, 249, 159, 224, 217, 30, 94,
64, 49, 115, 176, 155, 140, 115, 129, 243, 175, 218, 197, 292, 307, 253, 129, 55,
0, 81], [124, 122, 94, 101, 95, 115, 111, 153, 35, 38, 37, 62, 64, 135, 171, 179,
178, 191, 80, 147, 184, 75, 103, 19, 124, 34, 101, 97, 90, 74, 95, 190, 132, 194,
170, 246, 253, 187, 137, 80, 81, 0]], '_defaults': {'node_coords': {}, 'display_d
ata': {}, 'edge_data_format': None, 'demands': {}}, '_wfunc': <function StandardP
roblem._create_wfunc.<locals>.<lambda> at 0x000001B6F94756F0>, '_special': None}

```

In [7]: Aristas


```
Out[7]: [(0, 0),
(0, 1),
(0, 2),
(0, 3),
(0, 4),
(0, 5),
(0, 6),
(0, 7),
(0, 8),
(0, 9),
(0, 10),
(0, 11),
(0, 12),
(0, 13),
(0, 14),
(0, 15),
(0, 16),
(0, 17),
(0, 18),
(0, 19),
(0, 20),
(0, 21),
(0, 22),
(0, 23),
(0, 24),
(0, 25),
(0, 26),
(0, 27),
(0, 28),
(0, 29),
(0, 30),
(0, 31),
(0, 32),
(0, 33),
(0, 34),
(0, 35),
(0, 36),
(0, 37),
(0, 38),
(0, 39),
(0, 40),
(0, 41),
(1, 0),
(1, 1),
(1, 2),
(1, 3),
(1, 4),
(1, 5),
(1, 6),
(1, 7),
(1, 8),
(1, 9),
(1, 10),
(1, 11),
(1, 12),
(1, 13),
(1, 14),
(1, 15),
(1, 16),
(1, 17),
```

(1, 18),
(1, 19),
(1, 20),
(1, 21),
(1, 22),
(1, 23),
(1, 24),
(1, 25),
(1, 26),
(1, 27),
(1, 28),
(1, 29),
(1, 30),
(1, 31),
(1, 32),
(1, 33),
(1, 34),
(1, 35),
(1, 36),
(1, 37),
(1, 38),
(1, 39),
(1, 40),
(1, 41),
(2, 0),
(2, 1),
(2, 2),
(2, 3),
(2, 4),
(2, 5),
(2, 6),
(2, 7),
(2, 8),
(2, 9),
(2, 10),
(2, 11),
(2, 12),
(2, 13),
(2, 14),
(2, 15),
(2, 16),
(2, 17),
(2, 18),
(2, 19),
(2, 20),
(2, 21),
(2, 22),
(2, 23),
(2, 24),
(2, 25),
(2, 26),
(2, 27),
(2, 28),
(2, 29),
(2, 30),
(2, 31),
(2, 32),
(2, 33),
(2, 34),
(2, 35),

(2, 36),
(2, 37),
(2, 38),
(2, 39),
(2, 40),
(2, 41),
(3, 0),
(3, 1),
(3, 2),
(3, 3),
(3, 4),
(3, 5),
(3, 6),
(3, 7),
(3, 8),
(3, 9),
(3, 10),
(3, 11),
(3, 12),
(3, 13),
(3, 14),
(3, 15),
(3, 16),
(3, 17),
(3, 18),
(3, 19),
(3, 20),
(3, 21),
(3, 22),
(3, 23),
(3, 24),
(3, 25),
(3, 26),
(3, 27),
(3, 28),
(3, 29),
(3, 30),
(3, 31),
(3, 32),
(3, 33),
(3, 34),
(3, 35),
(3, 36),
(3, 37),
(3, 38),
(3, 39),
(3, 40),
(3, 41),
(4, 0),
(4, 1),
(4, 2),
(4, 3),
(4, 4),
(4, 5),
(4, 6),
(4, 7),
(4, 8),
(4, 9),
(4, 10),
(4, 11),

(4, 12),
(4, 13),
(4, 14),
(4, 15),
(4, 16),
(4, 17),
(4, 18),
(4, 19),
(4, 20),
(4, 21),
(4, 22),
(4, 23),
(4, 24),
(4, 25),
(4, 26),
(4, 27),
(4, 28),
(4, 29),
(4, 30),
(4, 31),
(4, 32),
(4, 33),
(4, 34),
(4, 35),
(4, 36),
(4, 37),
(4, 38),
(4, 39),
(4, 40),
(4, 41),
(5, 0),
(5, 1),
(5, 2),
(5, 3),
(5, 4),
(5, 5),
(5, 6),
(5, 7),
(5, 8),
(5, 9),
(5, 10),
(5, 11),
(5, 12),
(5, 13),
(5, 14),
(5, 15),
(5, 16),
(5, 17),
(5, 18),
(5, 19),
(5, 20),
(5, 21),
(5, 22),
(5, 23),
(5, 24),
(5, 25),
(5, 26),
(5, 27),
(5, 28),
(5, 29),

(5, 30),
(5, 31),
(5, 32),
(5, 33),
(5, 34),
(5, 35),
(5, 36),
(5, 37),
(5, 38),
(5, 39),
(5, 40),
(5, 41),
(6, 0),
(6, 1),
(6, 2),
(6, 3),
(6, 4),
(6, 5),
(6, 6),
(6, 7),
(6, 8),
(6, 9),
(6, 10),
(6, 11),
(6, 12),
(6, 13),
(6, 14),
(6, 15),
(6, 16),
(6, 17),
(6, 18),
(6, 19),
(6, 20),
(6, 21),
(6, 22),
(6, 23),
(6, 24),
(6, 25),
(6, 26),
(6, 27),
(6, 28),
(6, 29),
(6, 30),
(6, 31),
(6, 32),
(6, 33),
(6, 34),
(6, 35),
(6, 36),
(6, 37),
(6, 38),
(6, 39),
(6, 40),
(6, 41),
(7, 0),
(7, 1),
(7, 2),
(7, 3),
(7, 4),
(7, 5),

(7, 6),
(7, 7),
(7, 8),
(7, 9),
(7, 10),
(7, 11),
(7, 12),
(7, 13),
(7, 14),
(7, 15),
(7, 16),
(7, 17),
(7, 18),
(7, 19),
(7, 20),
(7, 21),
(7, 22),
(7, 23),
(7, 24),
(7, 25),
(7, 26),
(7, 27),
(7, 28),
(7, 29),
(7, 30),
(7, 31),
(7, 32),
(7, 33),
(7, 34),
(7, 35),
(7, 36),
(7, 37),
(7, 38),
(7, 39),
(7, 40),
(7, 41),
(8, 0),
(8, 1),
(8, 2),
(8, 3),
(8, 4),
(8, 5),
(8, 6),
(8, 7),
(8, 8),
(8, 9),
(8, 10),
(8, 11),
(8, 12),
(8, 13),
(8, 14),
(8, 15),
(8, 16),
(8, 17),
(8, 18),
(8, 19),
(8, 20),
(8, 21),
(8, 22),
(8, 23),

(8, 24),
(8, 25),
(8, 26),
(8, 27),
(8, 28),
(8, 29),
(8, 30),
(8, 31),
(8, 32),
(8, 33),
(8, 34),
(8, 35),
(8, 36),
(8, 37),
(8, 38),
(8, 39),
(8, 40),
(8, 41),
(9, 0),
(9, 1),
(9, 2),
(9, 3),
(9, 4),
(9, 5),
(9, 6),
(9, 7),
(9, 8),
(9, 9),
(9, 10),
(9, 11),
(9, 12),
(9, 13),
(9, 14),
(9, 15),
(9, 16),
(9, 17),
(9, 18),
(9, 19),
(9, 20),
(9, 21),
(9, 22),
(9, 23),
(9, 24),
(9, 25),
(9, 26),
(9, 27),
(9, 28),
(9, 29),
(9, 30),
(9, 31),
(9, 32),
(9, 33),
(9, 34),
(9, 35),
(9, 36),
(9, 37),
(9, 38),
(9, 39),
(9, 40),
(9, 41),

(10, 0),
(10, 1),
(10, 2),
(10, 3),
(10, 4),
(10, 5),
(10, 6),
(10, 7),
(10, 8),
(10, 9),
(10, 10),
(10, 11),
(10, 12),
(10, 13),
(10, 14),
(10, 15),
(10, 16),
(10, 17),
(10, 18),
(10, 19),
(10, 20),
(10, 21),
(10, 22),
(10, 23),
(10, 24),
(10, 25),
(10, 26),
(10, 27),
(10, 28),
(10, 29),
(10, 30),
(10, 31),
(10, 32),
(10, 33),
(10, 34),
(10, 35),
(10, 36),
(10, 37),
(10, 38),
(10, 39),
(10, 40),
(10, 41),
(11, 0),
(11, 1),
(11, 2),
(11, 3),
(11, 4),
(11, 5),
(11, 6),
(11, 7),
(11, 8),
(11, 9),
(11, 10),
(11, 11),
(11, 12),
(11, 13),
(11, 14),
(11, 15),
(11, 16),
(11, 17),

(11, 18),
(11, 19),
(11, 20),
(11, 21),
(11, 22),
(11, 23),
(11, 24),
(11, 25),
(11, 26),
(11, 27),
(11, 28),
(11, 29),
(11, 30),
(11, 31),
(11, 32),
(11, 33),
(11, 34),
(11, 35),
(11, 36),
(11, 37),
(11, 38),
(11, 39),
(11, 40),
(11, 41),
(12, 0),
(12, 1),
(12, 2),
(12, 3),
(12, 4),
(12, 5),
(12, 6),
(12, 7),
(12, 8),
(12, 9),
(12, 10),
(12, 11),
(12, 12),
(12, 13),
(12, 14),
(12, 15),
(12, 16),
(12, 17),
(12, 18),
(12, 19),
(12, 20),
(12, 21),
(12, 22),
(12, 23),
(12, 24),
(12, 25),
(12, 26),
(12, 27),
(12, 28),
(12, 29),
(12, 30),
(12, 31),
(12, 32),
(12, 33),
(12, 34),
(12, 35),

(12, 36),
(12, 37),
(12, 38),
(12, 39),
(12, 40),
(12, 41),
(13, 0),
(13, 1),
(13, 2),
(13, 3),
(13, 4),
(13, 5),
(13, 6),
(13, 7),
(13, 8),
(13, 9),
(13, 10),
(13, 11),
(13, 12),
(13, 13),
(13, 14),
(13, 15),
(13, 16),
(13, 17),
(13, 18),
(13, 19),
(13, 20),
(13, 21),
(13, 22),
(13, 23),
(13, 24),
(13, 25),
(13, 26),
(13, 27),
(13, 28),
(13, 29),
(13, 30),
(13, 31),
(13, 32),
(13, 33),
(13, 34),
(13, 35),
(13, 36),
(13, 37),
(13, 38),
(13, 39),
(13, 40),
(13, 41),
(14, 0),
(14, 1),
(14, 2),
(14, 3),
(14, 4),
(14, 5),
(14, 6),
(14, 7),
(14, 8),
(14, 9),
(14, 10),
(14, 11),

(14, 12),
(14, 13),
(14, 14),
(14, 15),
(14, 16),
(14, 17),
(14, 18),
(14, 19),
(14, 20),
(14, 21),
(14, 22),
(14, 23),
(14, 24),
(14, 25),
(14, 26),
(14, 27),
(14, 28),
(14, 29),
(14, 30),
(14, 31),
(14, 32),
(14, 33),
(14, 34),
(14, 35),
(14, 36),
(14, 37),
(14, 38),
(14, 39),
(14, 40),
(14, 41),
(15, 0),
(15, 1),
(15, 2),
(15, 3),
(15, 4),
(15, 5),
(15, 6),
(15, 7),
(15, 8),
(15, 9),
(15, 10),
(15, 11),
(15, 12),
(15, 13),
(15, 14),
(15, 15),
(15, 16),
(15, 17),
(15, 18),
(15, 19),
(15, 20),
(15, 21),
(15, 22),
(15, 23),
(15, 24),
(15, 25),
(15, 26),
(15, 27),
(15, 28),
(15, 29),

(15, 30),
(15, 31),
(15, 32),
(15, 33),
(15, 34),
(15, 35),
(15, 36),
(15, 37),
(15, 38),
(15, 39),
(15, 40),
(15, 41),
(16, 0),
(16, 1),
(16, 2),
(16, 3),
(16, 4),
(16, 5),
(16, 6),
(16, 7),
(16, 8),
(16, 9),
(16, 10),
(16, 11),
(16, 12),
(16, 13),
(16, 14),
(16, 15),
(16, 16),
(16, 17),
(16, 18),
(16, 19),
(16, 20),
(16, 21),
(16, 22),
(16, 23),
(16, 24),
(16, 25),
(16, 26),
(16, 27),
(16, 28),
(16, 29),
(16, 30),
(16, 31),
(16, 32),
(16, 33),
(16, 34),
(16, 35),
(16, 36),
(16, 37),
(16, 38),
(16, 39),
(16, 40),
(16, 41),
(17, 0),
(17, 1),
(17, 2),
(17, 3),
(17, 4),
(17, 5),

(17, 6),
(17, 7),
(17, 8),
(17, 9),
(17, 10),
(17, 11),
(17, 12),
(17, 13),
(17, 14),
(17, 15),
(17, 16),
(17, 17),
(17, 18),
(17, 19),
(17, 20),
(17, 21),
(17, 22),
(17, 23),
(17, 24),
(17, 25),
(17, 26),
(17, 27),
(17, 28),
(17, 29),
(17, 30),
(17, 31),
(17, 32),
(17, 33),
(17, 34),
(17, 35),
(17, 36),
(17, 37),
(17, 38),
(17, 39),
(17, 40),
(17, 41),
(18, 0),
(18, 1),
(18, 2),
(18, 3),
(18, 4),
(18, 5),
(18, 6),
(18, 7),
(18, 8),
(18, 9),
(18, 10),
(18, 11),
(18, 12),
(18, 13),
(18, 14),
(18, 15),
(18, 16),
(18, 17),
(18, 18),
(18, 19),
(18, 20),
(18, 21),
(18, 22),
(18, 23),

(18, 24),
(18, 25),
(18, 26),
(18, 27),
(18, 28),
(18, 29),
(18, 30),
(18, 31),
(18, 32),
(18, 33),
(18, 34),
(18, 35),
(18, 36),
(18, 37),
(18, 38),
(18, 39),
(18, 40),
(18, 41),
(19, 0),
(19, 1),
(19, 2),
(19, 3),
(19, 4),
(19, 5),
(19, 6),
(19, 7),
(19, 8),
(19, 9),
(19, 10),
(19, 11),
(19, 12),
(19, 13),
(19, 14),
(19, 15),
(19, 16),
(19, 17),
(19, 18),
(19, 19),
(19, 20),
(19, 21),
(19, 22),
(19, 23),
(19, 24),
(19, 25),
(19, 26),
(19, 27),
(19, 28),
(19, 29),
(19, 30),
(19, 31),
(19, 32),
(19, 33),
(19, 34),
(19, 35),
(19, 36),
(19, 37),
(19, 38),
(19, 39),
(19, 40),
(19, 41),

(20, 0),
(20, 1),
(20, 2),
(20, 3),
(20, 4),
(20, 5),
(20, 6),
(20, 7),
(20, 8),
(20, 9),
(20, 10),
(20, 11),
(20, 12),
(20, 13),
(20, 14),
(20, 15),
(20, 16),
(20, 17),
(20, 18),
(20, 19),
(20, 20),
(20, 21),
(20, 22),
(20, 23),
(20, 24),
(20, 25),
(20, 26),
(20, 27),
(20, 28),
(20, 29),
(20, 30),
(20, 31),
(20, 32),
(20, 33),
(20, 34),
(20, 35),
(20, 36),
(20, 37),
(20, 38),
(20, 39),
(20, 40),
(20, 41),
(21, 0),
(21, 1),
(21, 2),
(21, 3),
(21, 4),
(21, 5),
(21, 6),
(21, 7),
(21, 8),
(21, 9),
(21, 10),
(21, 11),
(21, 12),
(21, 13),
(21, 14),
(21, 15),
(21, 16),
(21, 17),

(21, 18),
(21, 19),
(21, 20),
(21, 21),
(21, 22),
(21, 23),
(21, 24),
(21, 25),
(21, 26),
(21, 27),
(21, 28),
(21, 29),
(21, 30),
(21, 31),
(21, 32),
(21, 33),
(21, 34),
(21, 35),
(21, 36),
(21, 37),
(21, 38),
(21, 39),
(21, 40),
(21, 41),
(22, 0),
(22, 1),
(22, 2),
(22, 3),
(22, 4),
(22, 5),
(22, 6),
(22, 7),
(22, 8),
(22, 9),
(22, 10),
(22, 11),
(22, 12),
(22, 13),
(22, 14),
(22, 15),
(22, 16),
(22, 17),
(22, 18),
(22, 19),
(22, 20),
(22, 21),
(22, 22),
(22, 23),
(22, 24),
(22, 25),
(22, 26),
(22, 27),
(22, 28),
(22, 29),
(22, 30),
(22, 31),
(22, 32),
(22, 33),
(22, 34),
(22, 35),


```
(22, 36),  
(22, 37),  
(22, 38),  
(22, 39),  
(22, 40),  
(22, 41),  
(23, 0),  
(23, 1),  
(23, 2),  
(23, 3),  
(23, 4),  
(23, 5),  
(23, 6),  
(23, 7),  
(23, 8),  
(23, 9),  
(23, 10),  
(23, 11),  
(23, 12),  
(23, 13),  
(23, 14),  
(23, 15),  
(23, 16),  
(23, 17),  
(23, 18),  
(23, 19),  
(23, 20),  
(23, 21),  
(23, 22),  
(23, 23),  
(23, 24),  
(23, 25),  
(23, 26),  
(23, 27),  
(23, 28),  
(23, 29),  
(23, 30),  
(23, 31),  
(23, 32),  
(23, 33),  
...]
```

```
In [8]: #Comprobar el nombre del problema  
print(problem.name)  
  
#Comprobar el número de Nodos  
print(len(Nodos))  
  
#Comprobar los 10 primeros Nodos  
print(Nodos[:10])  
  
#Comprobar las 10 primeras Aristas  
print(Aristas[:10])
```

swiss42

42

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9)]
```

```

NOMBRE: swiss42
TIPO: TSP
COMENTARIO: 42 Staedte Schweiz (Fricker)
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 72 78 82 159 122 131 206 112 57 28 43 70 1
15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 5
30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18
23 23 11 0 11 48 26 54 70 94 69 75 75 84 84 89 92 89 54 78 99 141 111 109 89 89 11 11 11 54
32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35
55 40 57 48 40 0 23 55 96 123 78 75 36 36 66 66 63 95 34 34 137 174 156 129 224 90 15 59 75
33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54
37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 40 40 87 87 94 158 158 163 242 135 65 6
92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65
114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 11
92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 6
110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 9
96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79
90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95
74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 9

```

In [9]: *#Probamos algunas funciones del objeto problem*

#Distancia entre nodos

`problem.get_weight(0, 1)`

#Todas las funciones

#Documentación: <https://tsplib95.readthedocs.io/en/v0.6.1/modules.html>

`#dir(problem)`

Out[9]: 15

```

In [10]: import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import MDS # Multidimensional Scaling o Escalado Multidimensional

```

```
def plot_tsp_solution(distance_matrix, tsp_solution):
```

```
    """
```

Dibuja el grafo de un TSP con las posiciones calculadas mediante MDS y muestra solo las aristas correspondientes a la solución del TSP.

:param distance_matrix: np.ndarray, matriz de distancias entre nodos

:param tsp_solution: list, lista de nodos en el orden de la solución del TSP

```
    """
```

```
    # Crear el grafo completo
```

```
    G = nx.Graph()
```

```
    num_nodes = len(distance_matrix)
```

```
    for i in range(num_nodes):
```

```
        for j in range(i + 1, num_nodes):
```

```
            G.add_edge(i, j, weight=distance_matrix[i][j])
```

```
    # Usar MDS para calcular posiciones de los nodos
```

```
    mds = MDS(n_components=2, dissimilarity="precomputed", random_state=42)
```

```
    positions = mds.fit_transform(distance_matrix)
```

```
    # Convertir las posiciones en un diccionario para networkx
```

```
    pos = {i: positions[i] for i in range(num_nodes)}
```

```
    # Crear un subgrafo con las aristas del camino TSP
```

```

TSP_G = nx.Graph()
for i in range(len(tsp_solution) - 1):
    u = tsp_solution[i]
    v = tsp_solution[i + 1]
    TSP_G.add_edge(u, v, weight=distance_matrix[u][v])

# Dibujar el grafo
plt.figure(figsize=(8, 6))

# Dibujar nodos
nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=500)

# Dibujar las aristas del camino TSP
nx.draw_networkx_edges(TSP_G, pos, edge_color='red', width=2)

# Añadir etiquetas a los nodos y pesos de las aristas
nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
nx.draw_networkx_edge_labels(TSP_G, pos, edge_labels=edge_labels, font_size=8)

plt.title("Grafo solucion TSP")
plt.show()

```

Funcionas basicas

```

In [11]: #Funcionas basicas
#####

#Se genera una solucion aleatoria con comienzo en en el nodo 0
def crear_solucion(Nodos):
    solucion = [Nodos[0]]
    for n in Nodos[1:]:
        solucion = solucion + [random.choice(list(set(Nodos) - set((Nodos[0])) - set(solucion)))]
    return solucion

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
    return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
    distancia_total = 0
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i],solucion[i+1], problem)
    return distancia_total + distancia(solucion[len(solucion)-1],solucion[0], problem)

sol_temporal = crear_solucion(Nodos)

distancia_total(sol_temporal, problem), sol_temporal

```

```
Out[11]: (4478,
          [0,
            27,
            10,
            16,
            29,
            18,
            19,
            20,
            35,
            12,
            13,
            8,
            14,
            37,
            4,
            5,
            31,
            3,
            32,
            9,
            1,
            38,
            40,
            21,
            39,
            26,
            24,
            28,
            33,
            30,
            36,
            15,
            23,
            7,
            25,
            11,
            22,
            41,
            2,
            6,
            34,
            17])
```

BUSQUEDA ALEATORIA

```
In [12]: # Elimina mensajes de Advertencias sobre versiones futuras
import warnings
warnings.filterwarnings('ignore', category=FutureWarning)
```

```
In [ ]: #####
# BUSQUEDA ALEATORIA
#####

def busqueda_aleatoria(problem, N):
    #N es el numero de iteraciones
    Nodos = list(problem.get_nodes())
```

```

mejor_solucion = []
#mejor_distancia = 10e100 #Inicializamos con un valor alto
mejor_distancia = float('inf') #Inicializamos con un valor alto

for i in range(N): #Criterio de parada: repetir N veces per
    solucion = crear_solucion(Nodos) #Genera una solucion aleatoria
    distancia = distancia_total(solucion, problem) #Calcula el valor objetivo(distancia total)

    if distancia < mejor_distancia: #Compara con la mejor obtenida hasta ah
        mejor_solucion = solucion
        mejor_distancia = distancia

print("Mejor solución:" , mejor_solucion)
print("Distancia :" , mejor_distancia)
return mejor_solucion

#Busqueda aleatoria con 10000 iteraciones
solucion = busqueda_aleatoria(problem, 10000)

plot_tsp_solution(problem.edge_weights, solucion)

```

BUSQUEDA LOCAL

```

In [ ]: #####
# BUSQUEDA LOCAL
#####
def genera_vecina(solucion):
    #Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nodos se generan (N-1)x(N-1)
    #Se puede modificar para aplicar otros generadores distintos que 2-opt
    #print(solucion)
    mejor_solucion = []
    mejor_distancia = 10e100
    for i in range(1, len(solucion)-1): #Recorremos todos los nodos en bucle doble para ev
        for j in range(i+1, len(solucion)):

            #Se genera una nueva solución intercambiando los dos nodos i,j:
            # (usamos el operador + que para listas en python las concatena) : ej.: [1,2] + [3] = [1,2,3]
            vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]

            #Se evalua la nueva solución ...
            distancia_vecina = distancia_total(vecina, problem)

            #... para guardarla si mejora las anteriores
            if distancia_vecina <= mejor_distancia:
                mejor_distancia = distancia_vecina
                mejor_solucion = vecina
    return mejor_solucion

#solucion = [1, 47, 13, 41, 40, 19, 42, 44, 37, 5, 22, 28, 3, 2, 29, 21, 50, 34, 30, 9, 16, 11, 38, 49, 10, 39, 3]
print("Distancia Solucion Inicial:" , distancia_total(solucion, problem))

```

```
nueva_solucion = genera_vecina(solucion)
print("Distancia Mejor Solucion Local:", distancia_total(nueva_solucion, problem))
```

```
In [ ]: #Busqueda Local:
# - Sobre el operador de vecindad 2-opt(funcion genera_vecina)
# - Sin criterio de parada, se para cuando no es posible mejorar.
def busqueda_local(problem):
    mejor_solucion = []

    #Generar una solucion inicial de referencia(aleatoria)
    solucion_referencia = crear_solucion(Nodos)
    mejor_distancia = distancia_total(solucion_referencia, problem)

    iteracion=0 #Un contador para saber las iteraciones que hacemos
    while(1):
        iteracion +=1 #Incrementamos el contador
        #print('#',iteracion)

        #Obtenemos la mejor vecina ...
        vecina = genera_vecina(solucion_referencia)

        #... y la evaluamos para ver si mejoramos respecto a lo encontrado hasta el momento
        distancia_vecina = distancia_total(vecina, problem)

        #Si no mejoramos hay que terminar. Hemos llegado a un minimo local(según nuestro operador d
        if distancia_vecina < mejor_distancia:
            #mejor_solucion = copy.deepcopy(vecina) #Con copia profunda. Las copias en python son por
            mejor_solucion = vecina #Guarda la mejor solución encontrada
            mejor_distancia = distancia_vecina

        else:
            print("En la iteracion ", iteracion, ", la mejor solución encontrada es:" , mejor_solucion)
            print("Distancia : " , mejor_distancia)
            return mejor_solucion

    solucion_referencia = vecina

sol = busqueda_local(problem)

plot_tsp_solution(problem.edge_weights, sol)
```

SIMULATED ANNEALING

```
In [ ]: #####
# SIMULATED ANNEALING
#####

#Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
#Mejorable eligiendo otra forma de elegir una vecina.
def genera_vecina_aleatorio(solucion):

    #Se eligen dos nodos aleatoriamente
    i,j = sorted(random.sample( range(1,len(solucion)) , 2))
```

```
#Devuelve una nueva solución pero intercambiando los dos nodos elegidos al azar
return solucion[i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]
```

```
#Funcion de probabilidad para aceptar peores soluciones
```

```
def probabilidad(T,d):
    if random.random() < math.exp( -1*d / T) :
        return True
    else:
        return False
```

```
#Funcion de descenso de temperatura
```

```
def bajar_temperatura(T):
    return T*0.99
```

In []:

```
# Recocido Simulado (Simulated Annealing)
```

```
def recocido_simulado(problem, TEMPERATURA ):
```

```
    #problem = datos del problema
```

```
    #T = Temperatura
```

```
    solucion_referencia = crear_solucion(Nodos)
```

```
    distancia_referencia = distancia_total(solucion_referencia, problem)
```

```
    mejor_solucion = [] #x* del pseudocodigo
```

```
    mejor_distancia = 10e100 #F* del pseudocodigo
```

```
N=0
```

```
while TEMPERATURA > .0001:
```

```
    N+=1
```

```
    #Genera una solución vecina
```

```
    vecina = genera_vecina_aleatorio(solucion_referencia)
```

```
    #Calcula su valor(distancia)
```

```
    distancia_vecina = distancia_total(vecina, problem)
```

```
    #Si es la mejor solución de todas se guarda(siempre!!!)
```

```
    if distancia_vecina < mejor_distancia:
```

```
        mejor_solucion = vecina
```

```
        mejor_distancia = distancia_vecina
```

```
    #Si la nueva vecina es mejor se cambia
```

```
    #Si es peor se cambia según una probabilidad que depende de T y delta(distancia_referencia - dis
```

```
    if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, abs(distancia_referencia - distancia_vecina)):
```

```
        #solucion_referencia = copy.deepcopy(vecina)
```

```
        solucion_referencia = vecina
```

```
        distancia_referencia = distancia_vecina
```

```
    #Bajamos la temperatura
```

```
    TEMPERATURA = bajar_temperatura(TEMPERATURA)
```

```
    print("La mejor solución encontrada es " , end="")
```

```
    print(mejor_solucion)
```

```
    print("con una distancia total de " , end="")
```

```
    print(mejor_distancia)
```

```
    return mejor_solucion
```

```
sol = recocido_simulado(problem, 10000000)

plot_tsp_solution(problem.edge_weights, sol)
```

Actividades extra

Mejora de algoritmo de búsqueda local

- Multiarranque
- Búsqueda en entornos variables

Recocido simulado

- Mejora en la selección de vecinos

Mejora de algoritmo de búsqueda local

- Multiarranque
- Búsqueda en entornos variables

```
In [ ]: def genera_vecina(solucion):
    # Genera Soluciones vecinas usando 2-opt sistemático
    mejor_solucion = []
    mejor_distancia = 10e100
    for i in range(1, len(solucion)-1):
        for j in range(i+1, len(solucion)):
            vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j:]
            distancia_vecina = distancia_total(vecina, problem)
            if distancia_vecina <= mejor_distancia:
                mejor_distancia = distancia_vecina
                mejor_solucion = vecina
    return mejor_solucion

def busqueda_local(solucion_inicial, problem):
    # Búsqueda local estándar basada en el código original
    actual = solucion_inicial
    distancia_actual = distancia_total(actual, problem)

    while True:
        vecina = genera_vecina(actual)
        distancia_vecina = distancia_total(vecina, problem)

        if distancia_vecina < distancia_actual:
            actual = vecina
            distancia_actual = distancia_vecina
        else:
            break
    return actual
```



```

# =====
# 1.- MULTIARRANQUE **Multi-start**
# =====

def busqueda_local_multiarranque(problem, num_arranques=10):
    # Ejecuta la búsqueda local múltiples veces desde diferentes puntos de partida aleatorios
    Nodos = list(problem.get_nodes())
    mejor_global = None
    mejor_distancia_global = 10e100

    print(f"--- Inia la Búsqueda Local con Multiarranque ({num_arranques} intentos) ->")
    for i in range(num_arranques):
        # Genera solución inicial aleatoria distinta
        sol_inicial = Nodos.copy()
        random.shuffle(sol_inicial)

        # Aplicar búsqueda local desde ese punto
        sol_local = busqueda_local(sol_inicial, problem)
        dist_local = distancia_total(sol_local, problem)

        # Print
        print(f"Intento {i+1}: Distancia obtenida = {dist_local}")

        # Guarda la mejor *global*
        if dist_local < mejor_distancia_global:
            mejor_global = sol_local
            mejor_distancia_global = dist_local

    print(f"\nMejor distancia global con Multiarranque: {mejor_distancia_global}")
    return mejor_global

# =====
# 2.- BÚSQUEDA EN ENTORNOS VARIABLES ***VNS simplificado***
# =====

def perturbacion(solucion, intensidad=3):
    # Modifica aleatoriamente intercambiando varios pares cambiando el entorno
    nueva = solucion.copy()
    n = len(nueva)
    for _ in range(intensidad):
        i, j = random.randint(0, n-1), random.randint(0, n-1)
        nueva[i], nueva[j] = nueva[j], nueva[i]
    return nueva

def busqueda_en_entornos_variables(problem, max_iter=15):
    # Explora distintos entornos cambiando de solución cuando la búsqueda local se estanca.
    Nodos = list(problem.get_nodes())
    actual = Nodos.copy()
    random.shuffle(actual)

    mejor_solucion = actual
    mejor_distancia = distancia_total(mejor_solucion, problem)

    print(f"\n--- Comienza Búsqueda en Entornos Variables (VNS) ---")
    for iteracion in range(max_iter):
        # 1. Aplicar búsqueda local estándar
        local = busqueda_local(actual, problem)
        dist_local = distancia_total(local, problem)

```

```

    if dist_local < mejor_distancia:
        mejor_solucion = local
        mejor_distancia = dist_local
        print(f"Iteración {iteracion+1}: Mejora encontrada -> Distancia: {mejor_distancia}")

    # 2. Cambio de entorno (Perturbación - escape del mínimo local)
    intensidad_salto = random.randint(2, 5)
    actual = perturbacion(mejor_solucion, intensidad=intensidad_salto)

    print(f"Mejor distancia final con VNS: {mejor_distancia}")
    return mejor_solucion

# Ejecución ---->
# Ejecutar Multiarranque
sol_multi = busqueda_local_multiarranque(problem, num_arranques=5)
plot_tsp_solution(problem.edge_weights, sol_multi)

# Ejecutar Búsqueda en Entornos Variables (VNS)
sol_vns = busqueda_en_entornos_variables(problem, max_iter=10)
plot_tsp_solution(problem.edge_weights, sol_vns)

```

Recocido simulado Mejorado

- Mejora en la selección de vecinos

```

In [ ]: def genera_vecina_2opt(solucion):
    # Genera la solución vecina invirtiendo un subsegmento aleatorio (Operador 2-opt)
    n = len(solucion)
    i = random.randint(0, n - 1)
    j = random.randint(0, n - 1)

    # i y j deben ser distintos y ordenados
    if i > j:
        i, j = j, i
    if i == j:
        return solucion.copy()

    nueva_solucion = solucion.copy()
    # se invierte la sublista entre i y j (operación clásica 2-opt)
    nueva_solucion[i:j+1] = reversed(nueva_solucion[i:j+1])
    return nueva_solucion

def probabilidad(T, delta):
    # Calcula la probabilidad de aceptar una peor solución (Se basada en la temperatura)
    if T <= 0:
        return False
    # Fórmula estándar de Boltzmann (recocido simulado)
    return random.random() < math.exp(-delta / T)

def bajar_temperatura(T):
    # Reduce la temperatura mediante un factor de enfriamiento geométrico
    return T * 0.99

```

```

def recocido_simulado(problem, TEMPERATURA):
    # Obtiene la lista de nodos desde el problema de tsplib95
    Nodos = list(problem.get_nodes())

    # Solución inicial de referencia (aleatoria)
    solucion_referencia = Nodos.copy()
    random.shuffle(solucion_referencia)

    # Usa la función distancia_total ya implementada
    distancia_referencia = distancia_total(solucion_referencia, problem)

    mejor_solucion = solucion_referencia.copy()
    mejor_distancia = distancia_referencia

    N = 0
    while TEMPERATURA > 0.0001:
        N += 1

        # Generar una solución vecina estructurada usando 2-opt
        vecina = genera_vecina_2opt(solucion_referencia)

        # Calcula su valor (distancia) usando la misma función
        distancia_vecina = distancia_total(vecina, problem)

        # Si es la mejor solución de todas, se guarda siempre
        if distancia_vecina < mejor_distancia:
            mejor_solucion = vecina.copy()
            mejor_distancia = distancia_vecina

        # Criterio de aceptación: si mejora o pasa la prueba de probabilidad térmica
        delta = abs(distancia_referencia - distancia_vecina)
        if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, delta):
            solucion_referencia = vecina.copy()
            distancia_referencia = distancia_vecina

        # Se baja la temperatura gradualmente
        TEMPERATURA = bajar_temperatura(TEMPERATURA)

    print("La mejor solución encontrada es: ", mejor_solucion)
    print("Con una distancia total de: ", mejor_distancia)
    return mejor_solucion

# Ejecución del algoritmo con una temperatura inicial alta
sol = recocido_simulado(problem, 10000000)

# solución resultante
plot_tsp_solution(problem.edge_weights, sol)

```

In []: